SalaryNet
Teaching Guide

Build, save, serve and deploy a simple neural network
with PyTorch, Flask and Google Cloud

A hands-on classroom walkthrough

1. The big picture

We will teach a computer to guess a person's salary from their years of experience. We do this with a **neural network** — a small mathematical function whose numbers (called **weights**) are adjusted automatically until its guesses match reality.

The whole journey has four stages:

- **Train** the network in a Jupyter notebook and **save** it to a file.
- **Serve** it with a Flask web app so anyone can get a prediction in a browser.
- **Package** everything with a requirements file and push it to GitHub.
- **Deploy** it to a Google Cloud Compute Engine virtual machine (a rented computer in a data centre) by cloning from GitHub.

> **Analogy:** training is like a student practising past papers. Saving the model is like writing down everything learned. Serving is opening a help desk where others can ask questions. Deploying is moving that help desk into a building the whole world can visit.

Files in the project

File	What it does
`model.py`	Defines the network. Shared by the notebook and the app.
`train.ipynb`	Trains the network and saves salary_net.pth.
`salary_net.pth`	The saved, trained weights.
`app.py`	Flask web app that loads the model and predicts.
`requirements.txt`	The exact Python packages needed (torch, flask).
`README.md`	Short run/deploy instructions.

2. What the network looks like

Our network, **SalaryNet**, is as small as a useful network gets. It takes **one number in** (years of experience) and gives **one number out** (salary in thousands of rupees).

- **Input layer:** 1 number — years of experience.
- **Hidden layer:** 2 neurons. Each makes its own weighted guess.
- **ReLU activation:** turns any negative value into 0. This small kink is what lets the network bend its line and learn the 'salary rises faster later' pattern.
- **Output layer:** 1 number — the predicted salary.

> A **neuron** just computes *(input × weight)* + *bias*. 'Learning' means nudging those weights and biases until the output is close to the real salaries.

model.py

```
import torch
from torch import nn

class SalaryNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(1, 2)   # 1 input  -> 2 hidden
        self.relu   = nn.ReLU()         # activation
        self.output = nn.Linear(2, 1)   # 2 hidden -> 1 output

    def forward(self, x):
        h = self.hidden(x)   # weighted sums
        h = self.relu(h)     # activation
        return self.output(h)
```

The same file also has a `load_model()` helper that rebuilds the network and loads the saved weights — the app uses it.

3. Training the network (train.ipynb)

Open the notebook and run the cells top to bottom. Here is what each important cell does.

The data

```
X = torch.arange(0, 11).float().reshape(-1, 1)   # 0..10 years
y = torch.tensor([20,25,30,35,50,65,80,95,110,125,140]).float().reshape(-1,1)
```

X is the experience, y is the real salary in thousands. The network never sees the answer rule — it must discover it.

The training loop

```
loss_fn   = nn.MSELoss()                              # how wrong are we?
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)

for epoch in range(3000):
    prediction = model(X)                 # 1. guess
    loss = loss_fn(prediction, y)         # 2. measure the error
    optimizer.zero_grad()                 # 3. reset old gradients
    loss.backward()                       # 4. how to improve?
    optimizer.step()                      # 5. nudge the weights
```

These five steps repeat 3000 times. Each pass is one **epoch**. The **loss** (the error) shrinks as the network improves.

- **MSELoss** — the average squared gap between guess and truth.
- **Adam** — the optimizer; it decides how big a nudge each weight gets.
- **lr=0.02** — the learning rate, the size of each nudge.

Saving the trained model

```
torch.save(model.state_dict(), MODEL_PATH)   # -> salary_net.pth
```

> We save only the **weights** (the `state_dict`), not the whole Python object. The app rebuilds the same SalaryNet and pours these learned weights back in. This is the normal, portable way to ship a model.

4. Serving predictions with Flask (app.py)

A trained model sitting in a file helps no one. **Flask** is a tiny web framework that turns our model into a website and an API.

The model is loaded **once** when the server starts — not on every request — so predictions are fast.

```
model = load_model()            # once, at startup

def predict_salary(experience):
    x = torch.tensor([[float(experience)]])
    with torch.no_grad():                    # no training now, just predict
        return model(x).item() * 1000        # thousands -> rupees
```

Endpoints (the different web addresses)

Address	Method	Purpose
`/`	`GET`	A web page with a form.
`/predict`	`POST`	Reads the form, shows the prediction.
`/api/predict`	`POST`	JSON in, JSON out — for other programs.
`/health`	`GET`	Returns 'ok'. Lets the cloud check it is alive.

Try it on your own computer

```
python app.py                # open http://localhost:8080

# or ask the API from a terminal:
curl -X POST http://localhost:8080/api/predict \
     -H 'Content-Type: application/json' \
     -d '{"experience": 5}'
# -> {"experience": 5, "salary": 65000}
```

5. Packaging for deployment

Before sending the app to a server we must list exactly what it needs so the server can rebuild the same environment.

The virtual environment (venv)

A **venv** is a private box of Python packages for this one project, so it never clashes with other projects on the machine.

```
python3 -m venv venv              # create the box
venv/bin/pip install -r requirements.txt  # fill it from the list
venv/bin/python app.py            # run the app using that box
```

requirements.txt

```
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.8.0   # the neural network library (CPU build = smaller)
flask==3.1.0   # the web framework
```

> The first line tells pip to fetch the **CPU-only** build of PyTorch. It is much smaller than the default GPU build and is all we need on a small VM. Pinning exact versions (`==2.8.0`) means the server installs the *same* packages you tested with — no surprises.

Push it to GitHub

We put the project on GitHub so the VM can simply **clone** it. The trained `salary_net.pth` is committed too, so the server never has to retrain.

```
git init
git add .
git commit -m "SalaryNet app"
# create a PUBLIC repo and push (using the GitHub CLI):
gh repo create SimpleNeuralNetwork --public --source=. --push
```

A `.gitignore` keeps the big `venv/` folder out of the repo — the VM builds its own.

6. Deploying to a GCP Compute Engine VM

A **Compute Engine VM** is a computer you rent from Google that runs 24/7 in a data centre. We will create one, pull our code from GitHub, run it, and open it to the internet. Follow every step in order.

> **Two different terminals — do not mix them up.**
> • **Cloud Shell** — the >_ icon at the top-right of the Cloud Console. It runs as *you* and can change firewall rules and VM settings.
> • **VM SSH** — opened with the **SSH** button next to your VM. It runs *inside* the VM and is where you install and run the app. It *cannot* change firewall rules.

Step 1 — Create the VM

- Console → search **VM instances** → **Create instance**.
- **Name:** anything, e.g. `salarynet-vm`. **Region:** pick one near you (e.g. `asia-south1`, Mumbai).
- **Machine type:** series **E2**, type **e2-small** (2 GB RAM). Enough for this tiny model.
- **Boot disk: Debian** or **Ubuntu** (the commands below use `apt-get`).
- Under **Firewall**, tick **Allow HTTP traffic** and **Allow HTTPS traffic**.
- Click **Create**.

> Those tick-boxes open ports 80 and 443 only. Our app runs on port **8080**, so we must open that ourselves in Step 2 — and tie it to the VM in Step 3. This is the step most people miss.

Step 2 — Open port 8080 (in Cloud Shell)

Open **Cloud Shell** and create a firewall rule. It allows anyone (`0.0.0.0/0`) to reach port 8080 on any VM that carries the network tag `salarynet`.

```
gcloud compute firewall-rules create allow-8080 \
    --allow tcp:8080 \
    --target-tags salarynet \
    --source-ranges 0.0.0.0/0
```

> If you run this inside the **VM SSH** window instead you get *'Request had insufficient authentication scopes.'* — the VM is not allowed to change firewall rules. Use **Cloud Shell**.

Step 3 — Put the salarynet tag ON the VM (in Cloud Shell)

The firewall rule only applies to VMs that carry the tag. A brand-new VM does **not** have it, so we add it. First find the VM's zone, then tag it.

```
gcloud compute instances list        # note the ZONE column

gcloud compute instances add-tags YOUR_VM_NAME \
    --zone=YOUR_ZONE --tags=salarynet
```

This is the single most common reason the website won't open. The rule exists, the app runs — but with no matching tag on the VM, Google silently blocks port 8080. Check the VM's tags with:

```
gcloud compute instances describe YOUR_VM_NAME --zone=YOUR_ZONE
--format="value(tags.items)"
```

Step 4 — Install and run the app (in VM SSH)

Click the **SSH** button next to your VM. Then run these, one at a time. The `pip install` downloads PyTorch, so give it a minute or two.

```
sudo apt-get update && sudo apt-get install -y git python3-venv
git clone https://github.com/Laxminarayen/SimpleNeuralNetwork.git
cd SimpleNeuralNetwork

python3 -m venv venv
venv/bin/pip install -r requirements.txt
venv/bin/python app.py
```

When you see `Running on http://0.0.0.0:8080`, the app is live. Leave this window open (closing it stops the app). Stop the app any time with **Ctrl+C**.

A harmless *'Failed to initialize NumPy'* warning may appear — ignore it, the app still works.

Step 5 — Open it in your browser

- Find the VM's **External IP** in the VM instances list (e.g. 34.171.128.177).
- Visit `http://EXTERNAL_IP:8080`

> Use **http://**, not https (we set up no certificate), and keep the **:8080** on the end — browsers hide it otherwise. Use the **External** IP, not the internal `10.x.x.x` one.

To update later

```
cd SimpleNeuralNetwork
git pull
venv/bin/python app.py
```

> **Remember to STOP the VM when class is over** (VM instances → tick it → Stop) so it stops costing money. Starting it again later gives a *new* External IP.

7. Troubleshooting (problems we really hit)

"Port 8080 is in use by another program"

An older copy of the app is still running and holding the port. First see what it is:

```
sudo ss -ltnp 'sport = :8080'
```

Then stop it. Match the name you see in the output (`python` or `gunicorn`) and kill it:

```
sudo pkill -9 -f app.py        # if it is python app.py
sudo pkill -9 -f gunicorn      # if it is gunicorn
sudo fuser -k 8080/tcp         # last resort: free the port directly
```

I killed it but it keeps coming back (new PID each time)

Something is **restarting** it automatically — a leftover **systemd service** with `Restart=always`. Killing the process just makes the service launch a new one. Find and remove the service:

```
systemctl list-units --type=service | grep -i -E 'salary|gunicorn'
sudo systemctl stop salarynet
sudo systemctl disable salarynet
sudo rm /etc/systemd/system/salarynet.service
sudo systemctl daemon-reload
```

The app runs, but the website won't open

Almost always the firewall tag. Work through these in order:

- On the VM: `curl http://localhost:8080/health` should print `{"status":"ok"}`. If yes, the app is fine — it is a network issue.
- In Cloud Shell: confirm the VM has the **salarynet** tag (Step 3). If missing, add it. This was our actual bug.
- Confirm the firewall rule exists: `gcloud compute firewall-rules describe allow-8080`
- In the browser, use **http://** and the **:8080** suffix with the **External** IP.

"insufficient authentication scopes" on a gcloud command

You ran a `gcloud` command inside the **VM SSH** window. Firewall and tag commands must be run in **Cloud Shell** instead.

8. Recap & glossary

The journey, start to finish

- Define the network once in **model.py**.
- Train it in **train.ipynb**, saving **salary_net.pth**.
- Load and serve it with **app.py** (Flask), run as **python app.py**.
- List dependencies in **requirements.txt** and push to **GitHub**.
- On a **GCP VM**: open port 8080, **tag the VM salarynet**, clone, and run.

Glossary

Term	Meaning
Neuron	A tiny formula: input × weight + bias.
Weight / bias	The numbers the network learns.
Epoch	One full pass over all the training data.
Loss	A number measuring how wrong the guesses are; we shrink it.
Optimizer (Adam)	The method that adjusts the weights to reduce loss.
state_dict	A dictionary of the model's learned weights that we save.
Flask	A small Python framework for building websites and APIs.
Endpoint	A web address the app responds to, e.g. /predict.
venv	A private, per-project box of Python packages.
Compute Engine VM	A computer rented from Google Cloud, running all the time.
Cloud Shell	A terminal that runs as YOU; used for firewall and tag commands.
VM SSH	A terminal INSIDE the VM; used to install and run the app.
Firewall rule	A setting that allows network traffic to a port, e.g. 8080.
Network tag	A label on a VM; firewall rules apply to VMs with a matching tag.
External IP	The VM's public internet address, used in the browser.